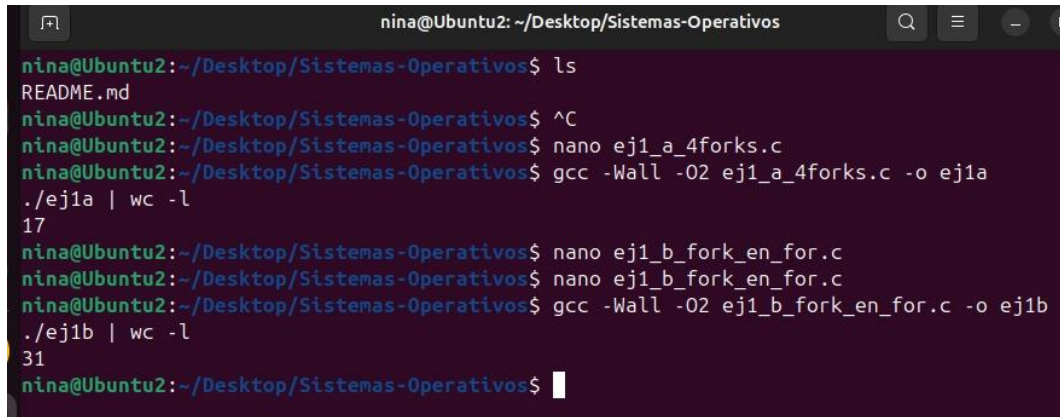


Ejercicio 1:



```
nina@Ubuntu2: ~/Desktop/Sistemas-Operativos
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ls
README.md
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ^C
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ nano ej1_a_4forks.c
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ gcc -Wall -O2 ej1_a_4forks.c -o ej1a
./ej1a | wc -l
17
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ nano ej1_b_fork_en_for.c
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ nano ej1_b_fork_en_for.c
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ gcc -Wall -O2 ej1_b_fork_en_for.c -o ej1b
./ej1b | wc -l
31
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$
```

- **Programa A**

Este programa ejecuta `fork()` cuatro veces seguidas. Cada `fork()` duplica el número de procesos existentes, por lo que al final se tienen $2^4 = 16$ procesos en total (incluyendo el proceso original).

En la ejecución se observa un total de 17 líneas impresas: 1 línea inicial “[Inicio]” (solo la imprime el proceso original antes de crear hijos) y 16 líneas (una por cada proceso que existe después de los 4 `fork()`).

- **Programa B**

Aunque aquí aparece un `fork()` dentro del ciclo, el detalle es que después del primer `fork()` existen dos procesos (padre e hijo) y ambos continúan ejecutando el `for`. En cada iteración, cada proceso vuelve a ejecutar `fork()`, por lo que el número de procesos se duplica por iteración: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$.

Al final también se tienen $2^4 = 16$ procesos. La diferencia es la cantidad de impresiones: dentro del `for` se imprime en cada iteración, por eso la salida total fue 31 líneas. Esas 31 se explican como 1 línea inicial “[Inicio]” + $(2 + 4 + 8 + 16)$ líneas del ciclo = $1 + 30 = 31$.

En ambos programas se crean 16 procesos porque en ambos casos se ejecutan 4 duplicaciones de procesos. En el primer caso las duplicaciones son explícitas (4 `fork()` seguidos). En el segundo caso, aunque el `fork()` está dentro de un `for`, el ciclo lo ejecutan el padre y todos los hijos, por lo que se vuelve a duplicar el número de procesos en cada iteración y se llega igualmente a 16.

Ejercicio 2:

```
nina@Ubuntu2: ~/Desktop/Sistemas-Operativos
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ gcc -Wall -O2 ej2_a_sin_concurrencia.c -o ej2a
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej2a
Tiempo (segundos): 0.001713
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej2a
Tiempo (segundos): 0.001380
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej2a
Tiempo (segundos): 0.001249
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej2a
Tiempo (segundos): 0.001671
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej2a
Tiempo (segundos): 0.001648
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej2a
Tiempo (segundos): 0.001500
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$
```

El programa corrió tres ciclos seguidos de 1,000,000 iteraciones cada uno. El tiempo lo medí usando clock() al inicio y al final. Lo ejecuté varias veces y los tiempos cambian un poquito entre cada ejecución.

```
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ nano ej2_b_con_forks.c
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ gcc -Wall -O2 ej2_b_con_forks.c -o ej2b
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej2b
Tiempo (segundos): 0.000084
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej2b
Tiempo (segundos): 0.000088
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej2b
Tiempo (segundos): 0.000076
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej2b
Tiempo (segundos): 0.000087
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej2b
Tiempo (segundos): 0.000154
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$
```

El programa corrió usando fork() para crear 4 procesos (padre, hijo, nieto y bisnieto). Cada proceso hizo un ciclo de 1,000,000 iteraciones y al final el padre esperó a los demás con wait() y midió el tiempo total con clock(). Lo ejecuté 5 veces y el tiempo cambia un poquito entre cada corrida.

En general, el programa sin forks debería ser más directo porque hace todo en un solo proceso. El programa con forks puede tardar más porque crear procesos (fork) y esperar procesos (wait) tiene costo extra, y también porque el sistema operativo tiene que repartir el CPU entre varios procesos (cambios de contexto). Aun así, si hay varios procesadores disponibles, algunas partes pueden ejecutarse al mismo tiempo y por eso a veces el tiempo sale parecido.

Ejercicio 3:

- **Cambios de contexto voluntarios:** pasan cuando un proceso cede el CPU por su cuenta, por ejemplo, cuando se queda esperando cuando hace `sleep()`, o cuando llama `wait()` para esperar a otro proceso.
- **Cambios de contexto involuntarios:** pasan cuando el sistema operativo le quita el CPU al proceso, por ejemplo, porque se le acabó su turno (time slice) o porque hay otro proceso con mayor prioridad que debe correr.

```
nina@Ubuntu2: ~/Desktop/Sistemas-Operativos
04:05 nina@Ubuntu2: $ ls
04:05 nina@Ubuntu2: $ ls
04:05 nina@Ubuntu2: $ cd Desktop/
04:05 nina@Ubuntu2:~/Desktop$ ls
04:05 nina@Ubuntu2:~/Desktop$ cd Sistemas-Operativos/
04:05 nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ls
04:05 ej1a ej1_b_fork_en_for.c ej2b
04:05 ej1a ej1_a_4forks.c ej2a ej2_b_con_forks.c
04:05 ej1b ej2_a_sin_concurrencia.c README.md
04:05 nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej2a
04:05 Tiempo (segundos): 0.001489
04:05 nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej2b
04:05 Tiempo (segundos): 0.006193
04:05 nina@Ubuntu2:~/Desktop/Sistemas-Operativos$
04:05
04:05
04:05
04:05
04:05:13 AM 0 30 1.97 0.00 kworker/0:0-
04:05:13 AM 0 47 1.98 0.00 kcompactd0
04:05:13 AM 990 405 0.99 0.00 systemd-oom
04:05:13 AM 0 1312 1.00 0.00 VBoxDRMClient
04:05:13 AM 1000 2216 13.04 7.94 gnome-shell
04:05:13 AM 1000 3093 7.92 0.00 chrome
04:05:13 AM 1000 3153 0.99 0.00 chrome
04:05:13 AM 1000 3386 18.81 0.00 chrome
04:05:13 AM 1000 4061 0.99 0.00 chrome
04:05:13 AM 1000 4112 11.00 0.00 gnome-terminal-
04:05:13 AM 0 4200 401.95 0.00 kworker/u1:1-events_power_efficient
04:05:13 AM 0 4254 7.92 0.00 kworker/u1:1-events
04:05:13 AM 0 4262 4.65 2.87 kworker/u1:3-events_unbound
04:05:13 AM 0 5043 1.98 0.00 kworker/u1:3-events_freezable_per_efficie
04:05:13 AM 0 5113 1.98 0.00 kworker/2:1-mm_percpu_wq
04:05:13 AM 0 5320 0.99 0.00 kworker/u1:4-Flush-BIO
04:05:13 AM 1000 10900 0.99 400.00 pidstat
```

Mientras corrí los programas del Ejercicio 2, observé en `pidstat` los cambios de contexto. Con el programa sin forks normalmente hay menos procesos involucrados. Con el programa con forks aparecen varios procesos (por padre/hijo/nieto/bisnieto) y se ven más cambios de contexto porque el CPU reparte tiempo entre ellos.

```
nina@Ubuntu2: ~/Desktop/Sistemas-Operativos
04:05:13 AM 0 30 1.97 0.00 kworker/0:0-
04:05:13 AM 0 47 1.98 0.00 kcompactd0
04:05:13 AM 990 405 0.99 0.00 systemd-oom
04:05:13 AM 0 1312 1.00 0.00 VBoxDRMClient
04:05:13 AM 1000 2216 13.04 7.94 gnome-shell
04:05:13 AM 1000 3093 7.92 0.00 chrome
04:05:13 AM 1000 3153 0.99 0.00 chrome
04:05:13 AM 1000 3386 18.81 0.00 chrome
04:05:13 AM 1000 4061 0.99 0.00 chrome
04:05:13 AM 1000 4112 11.00 0.00 gnome-terminal-
04:05:13 AM 0 4200 401.95 0.00 kworker/u1:1-events_power_efficient
04:05:13 AM 0 4254 7.92 0.00 kworker/u1:1-events
04:05:13 AM 0 4262 4.65 2.87 kworker/u1:3-events_unbound
04:05:13 AM 0 5043 1.98 0.00 kworker/u1:3-events_freezable_per_efficie
04:05:13 AM 0 5113 1.98 0.00 kworker/2:1-mm_percpu_wq
04:05:13 AM 0 5320 0.99 0.00 kworker/u1:4-Flush-BIO
04:05:13 AM 1000 10900 0.99 400.00 pidstat
Tiempo (segundos): 0.003697
nina@Ubuntu2:~/Desktop/Sistemas-Operativos
```

Se modificaron los programas para que impriman el valor del índice `i` mientras corren los ciclos. Al hacer prints, el programa tarda mucho más porque imprimir en pantalla es I/O y

eso hace que el sistema operativo tenga que cambiar más seguido entre procesos. En `pidstat` se nota que suben los cambios de contexto, especialmente los voluntarios, porque los procesos pasan tiempo esperando que la terminal imprima. Aparte que la compu se trabó y ya no respondía.

Ejercicio 4:

```
nina@Ubuntu2:~$ nano ej4_zombie.c
nina@Ubuntu2:~$ gcc -Wall -O2 ej4_zombie.c -o ej4z
nina@Ubuntu2:~$ ./ej4z
Hijo termina. PID=20056 | PPID=20055
Padre se queda en while(1). PID=20055
█
```

```
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ps -aef | head -n 5
ps -aef | grep defunct
F S  UID      PID     PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S   0        1        0  0  80   0 -  5746 -   ?           00:00:09 systemd
1 S   0        2        0  0  80   0 -    0 -   ?           00:00:00 kthreadd
1 S   0        3        2  0  80   0 -    0 -   ?           00:00:00 pool_workqueue_releas
e
1 I   0        4        2  0  60 -20 -    0 -   ?           00:00:00 kworker/R-rcu_gp
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$
```

El hijo terminó rápido, pero el padre se quedó corriendo en un `while(1)` y no hizo `wait()`. Por eso el hijo quedó como zombie: ya terminó, pero el sistema todavía guarda su entrada en la tabla de procesos porque el padre no lo recogió.

```
nina@Ubuntu2:~$ nano ej4_zombie.c
nina@Ubuntu2:~$ gcc -Wall -O2 ej4_zombie.c -o ej4z
nina@Ubuntu2:~$ ./ej4z
Hijo termina. PID=20056 | PPID=20055
Padre se queda en while(1). PID=20055
█
```

```
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ gcc -Wall -O2 ej4_huerfano.c -o ej4h
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej4h
Padre vivo (mátame con kill -9). PID=20432 | hijo PID=20433
Hijo PID=20433 contando... i=1000000 | PPID=20432
Hijo PID=20433 contando... i=2000000 | PPID=20432
Hijo PID=20433 contando... i=3000000 | PPID=20432
Hijo PID=20433 contando... i=4000000 | PPID=20432
Hijo terminó. PID=20433 | PPID=20432
```

```
nina@Ubuntu2: ~/Desktop/Sistemas-Operativos
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ps -ael | grep ej4h
0 R 1000 20432 19385 99 80 0 - 670 - pts/0 00:02:23 ej4h
1 Z 1000 20433 20432 0 80 0 - 0 - pts/0 00:00:00 ej4h
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$
```

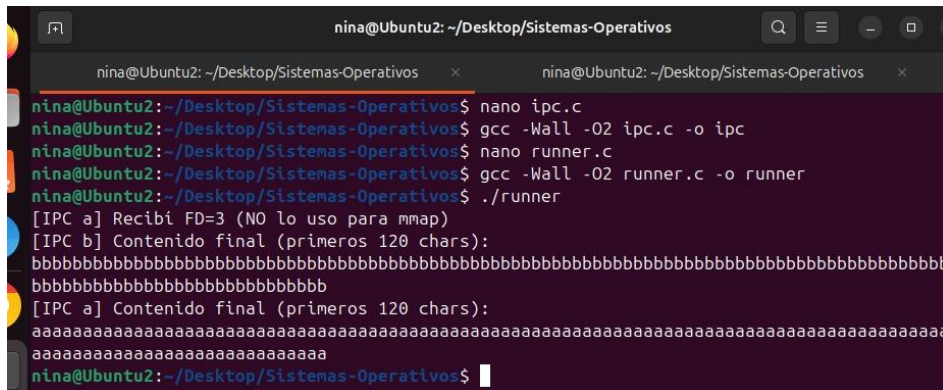
```
nina@Ubuntu2: ~/Desktop/Sistemas-Operativos
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ kill -9 20432
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ps -ael | grep ej4h
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$
```

```
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./ej4h
Padre vivo (mátame con kill -9). PID=20432 | hijo PID=20433
Hijo PID=20433 contando... i=1000000 | PPID=20432
Hijo PID=20433 contando... i=2000000 | PPID=20432
Hijo PID=20433 contando... i=3000000 | PPID=20432
Hijo PID=20433 contando... i=4000000 | PPID=20432
Hijo terminó. PID=20433 | PPID=20432
Killed
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$
```

Aquí el hijo ya terminó, pero el padre sigue corriendo y no hizo wait(). Por eso el hijo aparece como Z (zombie) y defunct: el proceso ya terminó, pero su entrada queda guardada hasta que el padre lo recoja.

Cuando maté al padre con kill -9, el proceso padre dejó de existir. Al desaparecer el padre, el zombie también deja de quedar colgado porque el sistema se encarga de limpiar ese proceso.

Ejercicio 5:



```
nina@Ubuntu2: ~/Desktop/Sistemas-Operativos
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ nano ipc.c
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ gcc -Wall -O2 ipc.c -o ipc
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ nano runner.c
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ gcc -Wall -O2 runner.c -o runner
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$ ./runner
[IPC a] Recibi FD=3 (NO lo uso para mmap)
[IPC b] Contenido final (primeros 120 chars):
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
[IPC a] Contenido final (primeros 120 chars):
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
nina@Ubuntu2:~/Desktop/Sistemas-Operativos$
```

Se implementó un programa ipc que recibe por parámetros un número n y un caracter x. El programa usa tres mecanismos de IPC: memoria compartida, un pipe ordinario y un pipe nombrado.

Se corrieron dos instancias del programa al mismo tiempo usando runner. Una instancia creó la memoria compartida y la otra detectó que ya existía. Para transferir el file descriptor de la shared memory entre instancias, se usó un socket local con sendmsg y SCM_RIGHTS, que es el método que sí permite pasar file descriptors reales entre procesos. La instancia que recibió el file descriptor lo imprimió, pero no se usó ese descriptor recibido para hacer mmap, sino que cada instancia hizo mmap usando su propio descriptor obtenido con shm_open.

Después de mapear la memoria, se hizo un fork() interno. En el proceso padre se recorrieron posiciones y cuando $i \% n == 0$ se envió una notificación por el pipe ordinario. En el proceso hijo se recibieron esas notificaciones y se escribió el caracter x en la memoria compartida. Al final, el padre esperó al hijo con wait() y se imprimió una parte del contenido final de la memoria compartida.

Con el output se observa que ambas instancias (a y b) escribieron sobre la misma memoria compartida, porque se imprimen resultados para los dos símbolos.

- **¿Qué diferencia hay entre realizar comunicación usando memoria compartida en lugar de usando un archivo de texto común y corriente?**

La memoria compartida permite que dos procesos vean y modifiquen los mismos bytes en RAM casi al instante. Un archivo de texto pasa por el sistema de archivos, suele ser más lento, y necesita lectura y escritura a disco o caché y más manejo de permisos y formato.

- **¿Por qué no se debe usar el file descriptor de la memoria compartida producido por otra instancia para realizar el mmap?**

Un file descriptor es válido solo dentro del proceso que lo tiene abierto. Si solo se recibe el número, en el otro proceso no apunta al mismo recurso. Para usarlo tendría

que transferirse como descriptor real con el mecanismo correcto, si no, mmap puede fallar o mapear algo incorrecto.

- **¿Es posible enviar el output de un programa ejecutado con exec a otro proceso por medio de un pipe? Investigue y explique cómo funciona este mecanismo en la terminal (e.g., la ejecución de ls | less).**

Sí es posible. La terminal conecta la salida estándar de un proceso al extremo de escritura del pipe y conecta la entrada estándar del otro proceso al extremo de lectura. Así, lo que normalmente se imprimiría en pantalla se convierte en entrada del siguiente programa, como en ls | less.

- **¿Cómo puede asegurarse de que ya se ha abierto un espacio de memoria compartida con un nombre determinado? Investigue y explique errno.**

Al abrir con shm_open usando O_CREAT y O_EXCL, si ya existe falla y errno queda en EEXIST. Si se recibe EEXIST, significa que el nombre ya estaba creado y se puede abrir sin O_EXCL.

- **¿Qué pasa si se ejecuta shm_unlink cuando hay procesos que todavía están usando la memoria compartida?**

Se elimina el nombre, pero la memoria compartida sigue existiendo mientras haya procesos que la tengan abierta o mapeada. Cuando el último proceso la cierra y la desmapea, el sistema la libera.

- **¿Cómo puede referirse al contenido de un espacio en memoria al que apunta un puntero? Observe que su programa deberá tener alguna forma de saber hasta dónde ha escrito su otra instancia en la memoria compartida para no escribir sobre ello.**

Se usa el puntero como un arreglo, por ejemplo, ptr[i] para leer o escribir un byte. Para no escribir encima de lo que ya escribió la otra instancia, se necesita un indicador compartido, como un índice guardado en la misma memoria, o un valor especial que marque hasta dónde hay datos.

- **Imagine que una ejecución de su programa sufre un error que termina la ejecución prematuramente, dejando el espacio de memoria compartida abierto y provocando que nuevas ejecuciones se queden esperando el file descriptor del espacio de memoria compartida. ¿Cómo puede liberar el espacio de memoria compartida “manualmente”?**

Se puede eliminar el nombre con shm_unlink usando el mismo nombre de la shared memory. También se puede borrar el archivo correspondiente en /dev/shm si quedó creado. Después de eso, una nueva ejecución puede crear el espacio limpio.

- **Observe que el programa que ejecute dos instancias de ipc.c debe cuidar que una instancia no termine mucho antes que la otra para evitar que ambas instancias**

abran y cierren su propio espacio de memoria compartida. ¿Aproximadamente cuánto tiempo toma la realización de un fork()? Investigue y aplique usleep.

Fork suele tardar desde microsegundos hasta pocos milisegundos, dependiendo del sistema y la carga. Para evitar que una instancia termine mucho antes, se puede agregar una espera corta con usleep, por ejemplo usleep de 50000 a 200000 microsegundos, para dar tiempo a que la otra instancia se conecte y haga su parte.